

I have uno doc

NegotiateAI

AI-Native Two-Sided Procurement Negotiation Platform

VisagioX Hackathon | Problem Statement #4 | April 2026

10 Hours Build Time	4 Members Team Size	180 Seconds Demo Slot	1 Database Supabase
---	---	---	---

This document is the internal team build reference. It is not for presenting to judges.

1. What We Are Building

NegotiateAI is a two-sided AI-native procurement platform. It sits between enterprise buyer companies and their vendors, giving both sides their own interface, their own AI agent, and connecting them through a live negotiation layer. The prototype must demonstrate the complete workflow from request to deal in a single unbroken demo.

1.1 The Core Innovation

KEY DIFFERENTIATOR Every procurement tool ever built serves only the buyer side. Vendors are always an afterthought — they receive emails and wait in silence. NegotiateAI is the first platform to give both parties equal tooling and AI representation in the same shared negotiation environment.

1.2 What the Prototype Must Demonstrate

The prototype must show five things in 180 seconds. These are non-negotiable:

1. An employee submits a purchase request in plain English — no form, no codes
2. AI parses the request into structured data and routes it for approval
3. A manager approves with one click — triggering the negotiation
4. A live AI negotiation chat — buyer AI generating real offers, vendor human responding, AI counter-offering intelligently
5. A deal is reached — both portals update, PO is generated

WHAT WE ARE CUTTING Email notifications → in-app only. PDF generation → success screen. Vendor AI agent → vendor human for the demo (simpler, equally compelling). Complex RLS multi-tenancy → two hardcoded demo accounts. Budget dashboard, benchmark layer, mobile view.

1.3 The Judging Criteria — How We Score

Resolves the Challenge	Complete end-to-end workflow from intake → approval → negotiation → PO. Passive SAP/Coupa replaced by system that actively negotiates. Directly answers: AI-native system of action replacing system of record.
Technically Innovative	Live AI negotiation agent using LLM reasoning with BATNA/ZOPA strategy. Supabase Realtime updating both portals simultaneously. n8n multi-step agentic workflow. Two-sided architecture with shared negotiation thread.
Practical	Natural language intake — no training needed. One-tap approval. Works on top of existing workflow without IT approval. Live working demo, not slides. Clear ROI: 15–30% savings on negotiated spend.

2. Technical Architecture

2.1 The Four Core Systems

Next.js + Vercel	Frontend for both buyer and vendor portals. Two separate routes (/buyer and /vendor) but one codebase. Deployed on Vercel — zero DevOps. Built with shadcn/ui + Tailwind.
Supabase	PostgreSQL database, Auth, and Realtime subscriptions. Single source of truth. All data lives here. Realtime is the live wire that makes both portals update instantly without page refresh.
n8n	Workflow automation engine. Handles: request parsing, approval routing, negotiation orchestration, AI agent loop. Leo's primary responsibility. Hosted on n8n Cloud.
OpenAI API	GPT-4o-mini for negotiation agent reasoning. GPT-4o-nano for request parsing and classification. Called by n8n at each workflow step. Estimated cost for demo: under \$2.

2.2 Database Schema

Six tables. Every table has **org_id** for buyer isolation and **vendor_id** for vendor isolation. For the prototype, we hardcode two demo orgs and skip full RLS — use simple WHERE clauses instead.

```
-- CORE TABLES
organizations (id, name, slug)
profiles      (id, org_id, vendor_id, role: admin|buyer|approver|vendor)

purchase_requests (id, org_id, requester_id, raw_text,
                    category, quantity, budget, timeline,
                    status: draft|pending_approval|approved|negotiating|complete)

approvals      (id, request_id, approver_id, decision, decided_at)

negotiations   (id, request_id, vendor_id, org_id,
                target_price, max_price, walkaway_price,
                current_offer_buyer, current_offer_vendor,
                round_count, status: active|agreed|escalated)

negotiation_messages (id, negotiation_id, sender_type,
                      sender_id, content, reasoning,
                      price_offered, terms_offered, created_at)
```

2.3 How Supabase Realtime Works in the Demo

THE MAGIC MOMENT The negotiation chat updating live on BOTH screens simultaneously — without either person refreshing — is the technical "wow" that judges remember. This is powered by Supabase Realtime subscriptions, which push database changes to the browser the instant they happen, like a live wire.

In the Next.js frontend, both portals subscribe to the negotiation_messages table:

```
// Both buyer AND vendor portals run this - each sees the same thread
const channel = supabase.channel('negotiation-' + negotiationId)
.on('postgres_changes', {
  event: 'INSERT',
  schema: 'public',
  table: 'negotiation_messages',
  filter: `negotiation_id=eq.${negotiationId}`
}, (payload) => {
  setMessages(prev => [...prev, payload.new]) // Appends new message
})
.subscribe()
```

2.4 n8n Negotiation Agent — The AI Loop

This is the most important workflow in the entire system. When a purchase request is approved, n8n runs this loop:

6. Webhook trigger fires when approval status changes in Supabase
7. n8n reads full request details + buyer parameters from Supabase
8. Creates negotiation row + sends opening offer message to DB (Supabase Realtime fires → both portals update)
9. n8n WAIT node — pauses until vendor responds (webhook from vendor's "Send" button)
10. n8n reads vendor message, builds full conversation history
11. OpenAI GPT-4o-mini called with system prompt containing BATNA/ZOPA strategy + conversation history
12. AI generates counter-offer + reasoning — n8n writes to negotiation_messages (Realtime fires again)
13. Loop repeats from step 4 until: deal found, max rounds hit, or guardrail triggered
14. On agreement: update negotiation status → create purchase_order row → both portals show completion

```
// n8n OpenAI system prompt (simplified)
You are a professional procurement negotiation agent.
Your parameters:
  Target price: {{$json.target_price}}
  Maximum price: {{$json.max_price}}
  Walk-away price: {{$json.walkaway_price}}
  Required terms: {{$json.required_terms}}
  Market benchmark: {{$json.benchmark_price}}

Negotiation history: {{$json.message_history}}

Vendor just offered: {{$json.vendor_offer}}

Respond with JSON:
{ "action": "counter|accept|escalate",
  "price": number,
  "message": "professional negotiation message",
  "reasoning": "internal reasoning for audit trail" }
```

```
RULES: Never exceed max_price. If vendor holds at walk-away  
for 2 rounds, set action to "escalate". Be professional.
```

3. The 180-Second Demo Script

CRITICAL RULE Pre-seed ALL demo data before presenting. Never demo from a blank state. Have 3 vendors already in the system, a sample org set up, and the approval workflow already tested end-to-end at least 5 times before the pitch slot.

3.1 Pre-Demo Setup (the night before / morning of)

- Create demo buyer org: "Acme Manufacturing" with 2 users — buyer (Sarah) and approver (Manager)
- Create demo vendor account: "OfficeSupplyCo" (this is the vendor portal your teammate operates live during demo)
- Seed negotiation parameters: target \$180/unit, max \$210/unit, walk-away \$220/unit, 500 office chairs
- Have vendor portal open on a second laptop or second browser window — visible to judges if possible
- Test the full flow at least 5 times — AI response times must be predictable
- Have a backup screen recording of a completed negotiation in case of live failure

3.2 The 180-Second Script

Presenter speaks. One teammate operates buyer portal. One teammate operates vendor portal.

0:00–0:15	THE HOOK — spoken while title slide shows "Procurement is broken. A company needs 500 chairs. Six weeks later, after 40 emails and 3 approval loops, they have a PO. We built the system that replaces that — in 12 hours."
0:15–0:40	REQUEST INTAKE — live on screen Buyer teammate types: "I need 500 office chairs, budget around \$90K, needed by end of month for our new Sydney office." Hit submit. Show the AI-parsed card appear — category, budget, quantity extracted. Presenter: "Natural language in. Structured data out. No forms."
0:40–1:00	APPROVAL ROUTING — live on screen Show manager notification appear in-app. Manager clicks Approve with one tap. Dashboard updates — status changes from Pending to Negotiating. Presenter: "Approved. AI agent activates."
1:00–2:30	THE AI NEGOTIATION — this is the 90-second money shot Buyer AI sends opening offer. Show it appear on BOTH screens simultaneously. Vendor teammate reads it, types a counter. Show buyer AI thinking (loading indicator). AI response appears on both screens with reasoning panel visible. Do 3 rounds. Presenter

narrates: "The AI is calculating its position using market benchmarks. It knows the target, the walk-away, and the full negotiation history." Round 3: AI accepts. Both screens show "Agreement Reached."

2:30–2:50

PO GENERATION
Click "Generate PO". PO confirmation card appears on both buyer and vendor portals simultaneously. Presenter: "Purchase order issued. Vendor confirmed. Done. What took 6 weeks took 4 minutes."

2:50–3:00

THE CLOSE
"NegotiateAI. The first procurement platform built for AI from day one — where both sides of every deal have intelligence working for them."

IF SOMETHING BREAKS Switch immediately to the backup screen recording. Have it queued and ready. Presenter continues speaking over the video as if it is live. Judges have seen live demos fail — a smooth recovery is fine. Never apologise excessively, just keep moving.

4. Team Roles and Responsibilities

Work is divided so each member can build independently for the majority of the day. Integration points are clearly defined so merging work at Hours 6–7 is straightforward. Leo has the most experience and owns the most complex backend components.

Leo (You)	n8n workflow automation + AI agent + Supabase schema + system integration
Member 2	Buyer portal — Next.js frontend (intake form, dashboard, approval UI, negotiation chat)
Member 3	Vendor portal — Next.js frontend (vendor dashboard, negotiation chat, PO view)
Member 4	Supabase setup + Auth + demo data seeding + slides + demo coordination

INTEGRATION HANDSHAKE At Hour 6, Leo shares the final Supabase table schemas and n8n webhook URLs with Members 2 and 3. Members 2 and 3 update their API calls to use the real endpoints. Member 4 seeds demo data. This is the only hard dependency — everything before Hour 6 can be built with mocked data.

5. Leo — Detailed Build Plan

Leo

n8n Automation + AI Agent + Supabase Schema + Integration Lead

YOUR ROLE You own everything that happens between the database and the AI. Your work is the engine. The frontend members build the car body — but without your engine, nothing moves. Your most critical delivery is the n8n negotiation loop working reliably by Hour 6.

Hour 0–1: Supabase Setup

0:00–0:30

Create Supabase project + schema

- Go to supabase.com → New project → name: negotiateai-demo
- Create the 6 tables from Section 2.2 using the SQL editor
- Add indexes: purchase_requests(org_id, status), negotiations(status), negotiation_messages(negotiation_id, created_at)
- Enable Realtime on negotiation_messages table (Table Editor → Realtime toggle)

0:30–1:00

Create demo seed data + auth users

- Create two auth users via Supabase Auth: buyer@demo.com / vendor@demo.com (password: demo1234)
- Insert org row: { id: uuid, name: "Acme Manufacturing", slug: "acme" }
- Insert vendor row: { id: uuid, name: "OfficeSupplyCo" }
- Insert profile rows linking each auth user to their org/vendor
- Note down all UUIDs — share with Member 4 for seed data

Hour 1–3: n8n Workflow — Request Parsing + Approval

1:00–1:45

n8n Workflow 1: Parse purchase request

- Trigger: Webhook (POST) — called by Next.js form submit
- Step 1: OpenAI node — prompt: "Extract from this text: category, quantity, budget_usd, timeline_days, item_description. Return JSON only." Model: gpt-4o-mini
- Step 2: Supabase node — INSERT into purchase_requests with parsed fields + status: pending_approval
- Step 3: Supabase node — INSERT into approvals table with approver_id = hardcoded manager UUID
- Output: Return { request_id, status: "pending" } to frontend

1:45–2:30

n8n Workflow 2: Approval routing

- Trigger: Webhook (POST /approve) — called when manager clicks approve button
- Step 1: Supabase — UPDATE approvals SET decision="approved" WHERE id={approval_id}
- Step 2: Supabase — UPDATE purchase_requests SET status="approved"

- Step 3: Supabase — INSERT into negotiations (target_price: 180, max_price: 210, walkaway_price: 220)
- Step 4: HTTP Request node — POST to Workflow 3 webhook to start negotiation
- Output: Return { negotiation_id } to frontend

2:30–3:00**Test Workflows 1 and 2 end to end**

- Use Postman or curl to test both webhooks manually
- Verify rows appear correctly in Supabase Table Editor
- Note the webhook URLs — share with Member 2 at Hour 3

Hour 3–6: n8n Workflow — AI Negotiation Loop (most complex)**3:00–4:30****n8n Workflow 3: AI negotiation agent loop**

- Trigger: Webhook (POST /start-negotiation) — receives negotiation_id
- Step 1: Supabase — fetch full negotiation row (all parameters)
- Step 2: Set node — build opening offer (start at target_price × 0.95)
- Step 3: Supabase — INSERT negotiation_messages { sender_type: "buyer_ai", content: opening message, price_offered: opening_price, reasoning: "Opening at 5% below target to create negotiating room" }
- -- Supabase Realtime fires here → both portals update --
- Step 4: Wait node — pause execution (set to "wait for webhook" mode)
- Step 5: Webhook resumes when vendor sends message (/vendor-reply)

4:30–5:30**n8n AI reasoning step (the heart of the system)**

- Step 6: Supabase — fetch all messages for this negotiation (full history)
- Step 7: Code node — build message_history string from all rows
- Step 8: OpenAI node — call GPT-4o-mini with system prompt from Section 2.4
- Include: target_price, max_price, walkaway_price, benchmark (\$185), full history, vendor's latest offer
- Step 9: Code node — parse JSON response { action, price, message, reasoning }
- Step 10: IF node — if action === "accept": go to close flow. If "escalate": notify human. If "counter": continue
- Step 11: Supabase — INSERT new buyer_ai message with AI's counter-offer
- Step 12: Go back to Step 4 (Wait node) — loop continues

5:30–6:00**n8n deal close flow**

- When action === "accept": UPDATE negotiations SET status="agreed", final_price={price}
- INSERT purchase_orders row with all agreed terms
- INSERT final buyer_ai message "We accept your terms. Pending final approval."
- Return all webhook URLs to share with the team at Hour 6 sync

Hour 6–7: Integration Sync**6:00–7:00****Team integration checkpoint**

- Share with Member 2: webhook URLs for /parse-request and /approve
- Share with Member 3: webhook URL for /vendor-reply, negotiation_id format

- Share with Member 4: all demo UUIDs for seed data, test all flows together
- Fix any integration bugs — this is the most likely problem hour
- Run full end-to-end flow once with all 4 people watching

Hour 7–9: Polish + Fallback

7:00–8:00

Add AI reasoning panel data

- Ensure reasoning field is being saved to negotiation_messages correctly
- Test that Member 2's reasoning panel correctly reads this field
- Add BATNA/ZOPA labels to the negotiation row so frontend can display strategy visualisation

8:00–9:00

End-to-end testing + demo prep

- Run the full demo flow 5 times — practice with teammates
- Time each run — target under 4 minutes to allow buffer
- Record one clean backup run (screen record) as insurance
- Intentionally break something and practice recovery

9:00–10:00

Buffer — fix bugs that emerged in testing

- Common issues: n8n Wait node timing out, OpenAI returning malformed JSON (add try/catch in Code node)
- Supabase Realtime not firing (check table is enabled for Realtime)
- CORS errors from n8n webhook (add Access-Control-Allow-Origin: * header in n8n response)

6. Brian — Buyer Portal

Member 2

Buyer Frontend — Next.js (intake form, dashboard, approval UI, negotiation chat)

YOUR ROLE You build everything the buyer sees. For the first 6 hours, use hardcoded/mock data and fake webhook calls so you can build independently. At Hour 6, swap in Leo's real webhook URLs and Supabase connection. The buyer portal is the PRIMARY screen shown during the demo.

Setup: Create Next.js Project

```
npx create-next-app@latest negotiateai --typescript --tailwind --app
cd negotiateai
npx shadcn@latest init
npx shadcn@latest add button card badge input textarea table
npm install @supabase/supabase-js
npm install lucide-react
```

Hour 0–2: Request Intake Page (/buyer/new-request)

0:00–1:00	Build the intake form – Large textarea — placeholder: "Describe what you need in plain English..." – Submit button triggers POST to n8n webhook (use mock URL for now: console.log the payload) – On response: show a "Request submitted" card with parsed fields (category, quantity, budget, timeline) – Show a loading spinner while waiting for n8n to respond
1:00–2:00	Build the parsed result card – After submission, show a card: "Your request has been parsed" – Display: Category badge, Quantity, Budget, Timeline, Status: "Pending Approval" – Add a subtle animation when the card appears (CSS transition) – Show a "Notify manager" indicator

Hour 2–4: Buyer Dashboard (/buyer/dashboard)

2:00–3:30	Build the main dashboard layout – Top row: 4 stat cards — Active Requests, Pending Approvals, Active Negotiations, Completed This Month – Use hardcoded numbers for now: 3, 1, 2, 8 – Main table: list of purchase requests with columns: Item, Category, Budget, Status (badge), Created
-----------	--

	– Status badges: gray=draft, yellow=pending, blue=approved, green=negotiating, green-dark=complete – Clicking a row navigates to the negotiation view
3:30–4:00	Connect dashboard to Supabase – npm install @supabase/supabase-js – Create /lib/supabase.ts with createClient using env vars – Replace hardcoded data with real Supabase query: <code>SELECT * FROM purchase_requests WHERE org_id = DEMO_ORG_ID</code> – Add useEffect to fetch on mount

Hour 4–6: Negotiation Chat View (/buyer/negotiation/[id])

4:00–5:30	Build the negotiation chat UI – Split layout: left 2/3 = chat thread, right 1/3 = strategy panel – Chat thread: messages displayed like WhatsApp — buyer_ai messages on right (blue), vendor messages on left (gray) – Each message shows: sender label, message text, price offered (if any), timestamp – AI messages show a small robot icon. Human messages show a person icon. – Input box at bottom with Send button — for the approval button only (vendor uses separate portal) – Add "AI is thinking..." animated dots when awaiting AI response
5:30–6:00	Strategy panel (right sidebar) – Show 3 price indicators as a vertical bar/progress: Target (\$180), Current AI offer, Max (\$210) – Show round count: "Round 3 of 10" – Show latest AI reasoning text from the reasoning field in the most recent buyer_ai message – Show negotiation status badge

Hour 6–7: Supabase Realtime (the live update magic)

6:00–7:00	Connect chat to Supabase Realtime – In the negotiation page, add Supabase Realtime subscription on mount – Subscribe to: <code>postgres_changes</code> on <code>negotiation_messages</code> WHERE <code>negotiation_id</code> = current id – On new INSERT: append message to local state — messages appear instantly without refresh – Scroll to bottom automatically when new message appears – Test: have Leo insert a row manually in Supabase — it should appear on screen instantly
-----------	--

Hour 7–9: Approval UI + Polish

7:00–8:00	Approval notification panel
-----------	------------------------------------

	– On dashboard, show a banner when a request is "pending_approval" – "Awaiting manager approval — [Request name]" – If logged in as approver role: show Approve / Reject buttons – Clicking Approve calls POST to n8n /approve webhook with request_id – Show status update immediately (optimistic UI update)
8:00–9:00	Polish for demo – Ensure the intake → parsed card → approval → negotiation flow is smooth with no broken states – Add proper loading states everywhere — no blank screens – Check that the chat autoscrolls correctly – Run through the demo flow 3 times solo, then once with the team

7. Member 3 — Vendor Portal

Member 3

Vendor Frontend — Next.js (vendor dashboard, negotiation chat, PO confirmation view)

YOUR ROLE You build what the vendor sees. This is the creative differentiator — the thing nobody has built before. During the demo, you ARE the vendor. You sit at your laptop with the vendor portal open, read the buyer AI's offers, type your counter-offers, and show judges that both sides of the negotiation are happening live simultaneously.

SETUP NOTE You can start from the same Next.js project as Member 2 (in a /vendor route) or create a separate project. Recommend: same project, separate routes. Share the Supabase client setup. The data is the same database — just filtered to vendor_id instead of org_id.

Hour 0–2: Vendor Dashboard (/vendor/dashboard)

0:00–1:30

Build vendor dashboard layout

- Top row: 4 stat cards — Active Negotiations, Pending Responses, Won This Month, Avg Response Time
- Use hardcoded numbers for now: 3, 1, 4, "12 min"
- Main table: Active negotiations with columns: Buyer Company, Item, Their Latest Offer, Round, Status
- Highlight row in yellow/amber if "Pending your response"
- Clicking a row opens the negotiation chat

1:30–2:00

Connect to Supabase (vendor-filtered)

- Use same Supabase client setup
- Query: `SELECT * FROM negotiations WHERE vendor_id = DEMO_VENDOR_ID`
- Join with negotiation_messages to get latest offer per negotiation

Hour 2–5: Vendor Negotiation Chat (/vendor/negotiation/[id])

2:00–3:30

Build vendor chat UI

- Same layout principle as buyer chat but mirrored perspective
- Vendor messages appear on right (amber/orange), buyer AI messages on left (blue)
- Buyer AI messages are clearly labelled: "Acme Corp AI Agent" with a robot icon
- Show a disclosure banner at top: "You are negotiating with an AI agent. Human contact available."
- Input box at bottom for vendor to type and send responses

3:30–4:30	Build vendor response flow – When vendor types message and clicks Send: – 1. INSERT message into negotiation_messages { sender_type: "vendor_human", content: text, price_offered: parsed price } – 2. POST to n8n /vendor-reply webhook with { negotiation_id, message_id } – 3. n8n wakes up, runs AI reasoning, inserts buyer AI response – 4. Supabase Realtime fires — new message appears on BOTH screens – Show "AI is composing response..." indicator after vendor sends (disappears when AI message arrives)
4:30–5:00	Price parsing helper – When vendor types a counter, auto-detect the price from their text – Simple regex: find dollar amounts in the message string – Show detected price below input: "Detected price: \$193/unit — correct?" with Edit option – Store this as price_offered in the message row

Hour 5–6: Vendor Parameters Panel

5:00–6:00	Build vendor strategy sidebar – Right panel: show vendor's own private parameters (from negotiations table or hardcoded) – Floor price indicator: \$195 (your minimum) – Current offer from buyer: highlighted in blue – Gap indicator: "Current gap: \$8/unit" — updates as offers come in – Round counter and negotiation status – This panel is ONLY visible to the vendor — buyers cannot see it
-----------	--

Hour 6–7: Realtime + Integration

6:00–7:00	Supabase Realtime subscription (same pattern as buyer) – Subscribe to negotiation_messages for current negotiation_id – On INSERT: append message to state, scroll to bottom – Test with Leo: Leo inserts a buyer_ai message manually — verify it appears on vendor screen – Test the reverse: you type a vendor message — verify it appears on buyer screen (check with Member 2)
-----------	--

Hour 7–9: PO View + Demo Practice

7:00–8:00	PO confirmation view – When negotiation status = "agreed": show a full-screen success card on vendor portal – "Deal Agreed ✓" in large green text – Show agreed terms: price, quantity, total value, payment terms, delivery – Show "Purchase Order Issued" card with PO number (auto-generated: PO-2026-XXXX)
-----------	--

	– Show "Expected payment by [date]" — calculate based on net-45 from today
8:00–9:00	Demo practice as vendor operator – Practice your role: you are playing the vendor in the demo – Know exactly what counter-offers to type: start at \$194, move to \$191, accept at \$191 if AI pushes – Practice reading AI messages quickly and responding naturally – Time your responses — aim for 10–15 seconds between vendor messages to keep demo pace – Practice the moment the deal closes — know which screen to show

8. Member 4 — Supabase + Auth + Slides + Demo Coordination

Member 4

Supabase Configuration + Auth + Demo Data + Slides + Demo Coordinator

YOUR ROLE You are the glue. Your technical work (Supabase auth, seed data) enables everyone else to test against real data. Your non-technical work (slides, demo coordination) is what the judges actually see first. Balance these carefully — the slides need to be done by Hour 9 at the latest.

Hour 0–2: Supabase Auth Configuration

0:00–1:00	Configure Supabase Auth – In Supabase Auth settings: enable Email provider – Disable email confirmation (for demo — we don't want to wait for emails) – Create 4 auth users: buyer@demo.com, approver@demo.com, vendor@demo.com, admin@demo.com (all password: Demo1234!) – Note down all user UUIDs from the Auth dashboard — share with Leo
1:00–2:00	Seed demo data in Supabase – Wait for Leo to finish the schema (ask him to confirm tables exist by Hour 1) – INSERT org: { name: "Acme Manufacturing", slug: "acme" } – INSERT vendor: { name: "OfficeSupplyCo", contact_name: "James Wilson" } – INSERT profiles: link buyer, approver users to org; link vendor user to vendor record – INSERT 2-3 completed negotiations with fake data — makes the dashboard look real and active – INSERT a sample purchase request with status "pending_approval" — ready for demo

Hour 2–5: Slides

SLIDE PRIORITY	Slides are supplementary to the demo, not the main event. Keep them minimal. 5 slides maximum. They run in the background while the live demo happens — they are visual context, not the pitch.
-----------------------	---

2:00–3:30	Build the 5 slides – Slide 1 (TITLE): "NegotiateAI" large, subtitle "The first procurement platform built for AI from day one"
-----------	---

	– Slide 2 (PROBLEM): Two columns — "Today" (email chaos, 6 weeks, 40 messages) vs "The Gap" (stat: 50% still use spreadsheets, \$15/invoice manual cost) – Slide 3 (SOLUTION): Simple architecture diagram — Buyer Portal ↔ NegotiateAI Platform ↔ Vendor Portal. Show the 4 negotiation modes as a 2×2 grid. – Slide 4 (LIVE DEMO): Just the words "LIVE DEMO" and the two-portal split screen graphic. This slide shows while the demo runs. – Slide 5 (IMPACT): Three numbers — "15-30% cost savings" / "47 minutes vs 6 weeks" / "First AI-to-AI procurement platform"
3:30–5:00	Polish slides – Use a consistent dark navy/blue colour scheme matching the product – Keep text minimal — slides support what the presenter says, not repeat it – Add simple icons (from Flaticon or Lucide) — no stock photos – Export as PDF backup in case Keynote/PowerPoint has issues

Hour 5–7: Testing + Environment Variables

5:00–6:00	Create .env files for both portals – Create .env.local with Supabase URL and anon key for Member 2's project – Create .env.local for Member 3's project (same values) – Add n8n webhook URLs once Leo shares them at Hour 6 – Verify both portals can connect to Supabase by running locally
6:00–7:00	Integration testing support – During the Hour 6 sync: monitor Supabase Table Editor in real time – As Leo, Member 2, Member 3 test their flows, watch rows appearing in the tables – Flag any data issues (missing fields, wrong IDs, etc.) – Keep a text file of all test results — pass/fail for each step

Hour 7–10: Demo Coordination + Rehearsal

7:00–8:00	Demo logistics setup – Set up the two-laptop configuration: Buyer portal on presentation laptop, Vendor portal on second laptop – Both visible to judges if possible — or use split screen on one large monitor – Clear browser history, close all other tabs, set browser zoom to 110% for visibility – Confirm WiFi password at venue. Have mobile hotspot as backup. – Create a demo reset script: SQL to clear test messages and reset negotiation status
8:00–9:30	Full rehearsal runs – Run the complete demo 3 times with all 4 people in their roles – Time each run — should be under 3:30 with buffer for talking – After each run: note what broke, what looked good, what to improve – Assign a clear countdown signal (hand signal at 2:00 and 2:45 remaining)

9:30–10:00

Final preparation

- Reset demo data to clean state — run the reset SQL script
- Queue backup screen recording
- Print or have digital copy of the demo script for each presenter
- Confirm presenting order and who speaks when

9. Master Timeline — All Members

Hour	Leo	Member 2	Member 3	Member 4
0–1	Supabase setup + schema + seed users	Next.js + shadcn setup + intake form start	Next.js setup (same project) + vendor dashboard start	Supabase auth config + create users
1–2	n8n Workflow 1: request parsing	Intake form + parsed result card	Vendor dashboard table + Supabase query	Seed demo data into all tables
2–3	n8n Workflow 2: approval routing	Buyer dashboard layout + stat cards	Vendor negotiation chat UI layout	Slides: title + problem slides
3–4	n8n Workflow 3: negotiation loop (start)	Buyer dashboard Supabase connection	Vendor chat message display	Slides: solution + demo slides
4–5	n8n Workflow 3: AI reasoning step	Negotiation chat UI + strategy panel	Vendor response flow + price parsing	Slides: impact slide + polish
5–6	n8n deal close flow + test all webhooks	Supabase Realtime connection	Vendor parameters panel	.env files + Supabase connection tests
6–7	🔴 INTEGRATION SYNC — share URLs + fix bugs together			
7–8	AI reasoning panel polish + e2e testing	Approval UI + dashboard polish	PO success view	Demo laptop setup + logistics
8–9	Buffer: fix bugs from testing	Demo practice solo x3	Demo practice as vendor operator x3	Full rehearsal coordination
9–10	Record backup video	Final flow test	Final flow test	Reset demo data + final prep

INTEGRATION SYNC (Hour 6) This is the most critical coordination point of the day. Everyone stops their independent work and connects the pieces together. Leo shares all webhook URLs. Members 2 and 3 update their API calls. Member 4 verifies data. The system runs end-to-end for the first time. Budget 60 minutes for this — things will break.

10. Risk Register and Contingencies

n8n Wait node times out during demo	Increase timeout to 10 minutes. Test at venue WiFi. Have backup: trigger vendor reply manually via Supabase INSERT if needed.
OpenAI returns malformed JSON	Add try/catch in n8n Code node. Fallback: hardcode a response if parse fails. Test with 20 calls before demo day.
Supabase Realtime not updating	Check table is enabled for Realtime. Fallback: add a 3-second polling interval as backup. Realtime is usually reliable.
n8n Cloud webhook URL changes	Save webhook URLs immediately when created. Use n8n's stable production URLs not test URLs. Verify morning of demo.
WiFi failure at venue	Have mobile hotspot. n8n Cloud and Supabase are external services — needs internet. Brief team on hotspot switchover procedure.
Demo takes too long	Practice until consistently under 3:00. If going long: skip the strategy panel explanation, just say "AI reasoning is visible here" and move on.
Integration bugs at Hour 6	Budget the full hour. Common fix: CORS headers on n8n webhooks, wrong Supabase env vars, wrong UUID formats. Start with a simple ping test.
OpenAI API key runs out	Pre-fund with \$20 credit. Monitor usage during testing. Have a second API key from a different account as backup.

11. Quick Reference — Shared Resources

Environment Variables (fill in as created)

```
# .env.local — shared by both portals
NEXT_PUBLIC_SUPABASE_URL=           # From Supabase project settings
NEXT_PUBLIC_SUPABASE_ANON_KEY=      # From Supabase project settings

# n8n webhook URLs (Leo fills these in at Hour 1-2)
NEXT_PUBLIC_N8N_PARSE_REQUEST=      # POST with { raw_text }
NEXT_PUBLIC_N8N_APPROVE=            # POST with { request_id }
NEXT_PUBLIC_N8N_VENDOR_REPLY=       # POST with { negotiation_id, message }

# Demo account credentials
BUYER_EMAIL=buyer@demo.com           BUYER_PASS=Demo1234!
APPROVER_EMAIL=approver@demo.com     APPROVER_PASS=Demo1234!
VENDOR_EMAIL=vendor@demo.com         VENDOR_PASS=Demo1234!
```

Demo Reset SQL (run before each demo attempt)

```
-- Run this in Supabase SQL Editor to reset demo state
DELETE FROM negotiation_messages WHERE negotiation_id IN (
  SELECT id FROM negotiations WHERE created_at > NOW() - INTERVAL '1 hour'
);
DELETE FROM negotiations WHERE created_at > NOW() - INTERVAL '1 hour';
UPDATE purchase_requests SET status = 'pending_approval'
WHERE id = 'DEMO_REQUEST_UUID'; -- replace with actual UUID
```

Key Supabase Demo UUIDs (fill in as created)

Acme Org ID	_____ (fill when created)
OfficeSupplyCo Vendor ID	_____ (fill when created)
Buyer User ID	_____ (fill when created)
Approver User ID	_____ (fill when created)
Vendor User ID	_____ (fill when created)
Demo Request ID	_____ (fill when created)

Success Definition — What Does "Done" Look Like?

DONE MEANS An employee types a request → manager approves → buyer AI sends an offer → vendor types a counter → buyer AI responds intelligently with a counter-offer → deal is reached → both portals show "Agreement Reached" simultaneously. This complete flow works reliably 5 times in a row without errors. That is done.

